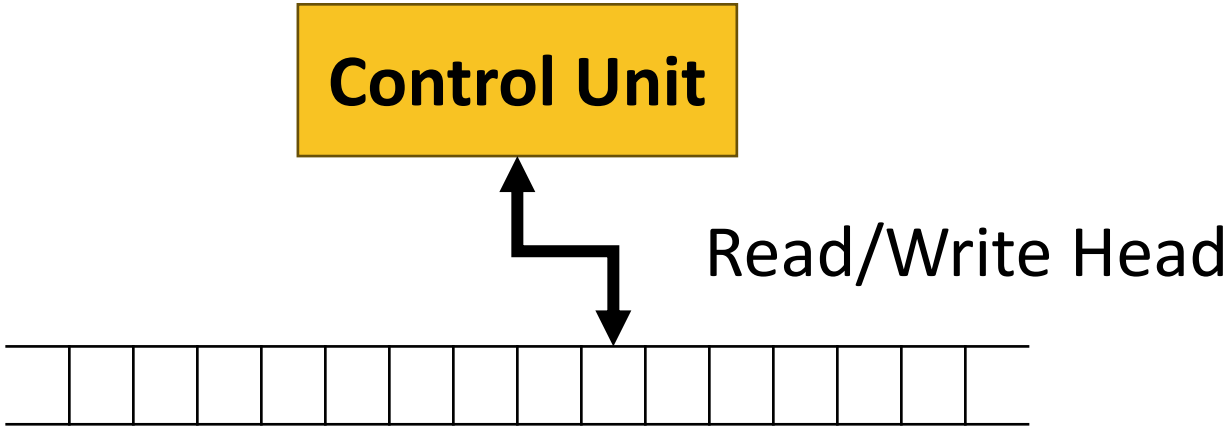
Turing Machines (3.1)

COT 4210 Discrete Structures II
Summer 2025
Department of Computer Science
Dr. Steinberg

The Turing Machine

- With DFAs and NFAs, we formalized the notion of decision-based computation
  - With PDAs, we provided a highly restricted memory for that computation
  - Now we define a type of machine with unrestricted memory
- A **Turing Machine** is like a finite automaton with an infinite **tape** that is used as memory
  - Input comes in on the tape; output is likewise written to the tape
  - Like a finite automaton, it can decide languages
  - Unlike a finite automaton, it can also produce other forms of output
- In general terms, at every step, a Turing machine can:
  - Transition based on the current state and the tape symbol at the current position, like a DFA
  - Write a symbol to the tape at the current position if it wishes
  - Move its **head** *either* left *or* right on the tape


Control Unit
Read/Write Head

Differences between Turing Machines and Finite Automata

- A Turing machine can write to its tape as well as reading from it
  - Generalized, a Turing machine is **read-write** rather than **read-only**
- A Turing machine's tape can move both to the left and to the right
  - Generalized, a Turing machine has **random-access** capability
- A Turing machine's tape is infinite
  - Generalized, a Turing machine has an **arbitrary amount of memory**
- There are special states for rejecting and accepting, that take effect immediately
  - A Turing machine's final states are **immediately final**

A Turing Machine for a Familiar Language

Let language *A* = { *w*#*w* | *w* in { **0**, **1** }* }

- Keep in mind the input size is unbounded
- We can't remember the whole string…
- …but since the alphabet is finite, we can remember any *character* by capturing it in the states

On input string *w*, begin at the leftmost symbol, and repeat the following:

1. Remember the symbol by state
2. Move the head to the corresponding symbol on the right of the #
   a. If the symbol on the right doesn't match the symbol on the left, or we find no #, **reject immediately**
3. Mark off the symbol on the right with an **x**
4. Move the head back to the symbol on the left, and mark it off with an **x**
5. Move the head to the next symbol on the left
6. If we've reached the #:
   a. If we have remaining symbols on the right that have not been marked off, **reject**
   b. Otherwise, **accept**
7. Otherwise, return to Step 1

The Machine In Operation

On input string *w*, begin at the leftmost symbol, and repeat the following:

1. Remember the symbol by state
2. Move the head to the corresponding symbol on the right of the #
   a. If the symbol on the right doesn't match the symbol on the left, or we find no #, **reject immediately**
3. Mark off the symbol on the right with an **x**
4. Move the head back to the symbol on the left, and mark it off with an **x**
5. Move the head to the next symbol on the left
6. If we've reached the #:
   a. If we have remaining symbols on the right that have not been marked off, **reject**
   b. Otherwise, **accept**
7. Otherwise, return to Step 1

Here we see some configurations of this machine on input 011000#011000.

0 1 1 0 0 0 # 0 1 1 0 0 0 □

0 1 1 0 0 0 # ***0*** 1 1 0 0 0 □

0 1 1 0 0 0 # x 1 1 0 0 0 □

x 1 1 0 0 0 # x ***1*** 1 0 0 0 □

x ***1*** 1 0 0 0 # x x 1 0 0 0 □

x x 1 0 0 0 # x x 1 0 0 0 □

x x 1 0 0 0 # x x ***1*** 0 0 0 □

…

x x x x x x # x x x x x x □

Definition: Turing Machine

- A **Turing machine** is a 7-tuple (Q, Σ, Γ, δ, q_0, q_{accept}, q_{reject}) with:
  - Q as a finite set of states,
  - Σ as the input alphabet not containing the blank symbol $\square$
  - Γ as the tape alphabet, which is a superset of Σ and *does* contain $\square$
  - δ: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ as the transition function,
  - q_0 as the start state,
  - q_{accept} as the accept state, and
  - q_{reject} as the reject state, with $q_{accept} \neq q_{reject}$.
- The heart of the machine is the transition function
  - Say that $\delta(q, a) = (r, b, L)$
  - Then on a state q and tape square containing symbol a, the machine will replace the a with b, transition to r, and move the tape head left

Turing Machine Notes and Terminology 1

- A Turing machine receives its input on the **leftmost squares** of the tape
- The infinite rest of the tape is blank (i.e., filled with the blank symbol)
- The head starts on the **leftmost square**
- The alphabet $\sum$ **does not contain the blank symbol**
- If the head tries to move **left from the leftmost square**, it **stays on the leftmost square instead**
- The **configuration** of a Turing machine is described by the current:

1. State
2. Contents of the tape
3. Location of the head

Turing Machine Notes and Terminology 2

- To **concisely represent** the configuration with tape contents uv in state q with the head over the first symbol of v, we can write:

$$u\ q\ v$$

- A configuration C_1 **yields** a configuration C_2 if a Turing machine can legally go from C_1 to C_2 in a single step
- The **start configuration** is always $q_0\ w$
- An **accepting configuration** is any configuration where the state is q_{accept}
- A **rejecting configuration** is any configuration where the state is q_{reject}
- Accepting and rejecting configurations are **halting configurations** that do not yield further configurations

A Bit More On Transitions

To represent the configuration with tape contents *uv*, in state *q*, with the head over the first symbol of *v*, we write: ***u q v***

We can clarify our transitions by letting *x* and *a* be the last character of the prefix and first character of the suffix, shortening *u* and *v* correspondingly, and observing the following—

ux q av yields:

- *u r xbv* if $\delta(q, a) = (r, b, L)$
- *uxb r v* if $\delta(q, a) = (r, b, R)$

--with:

- *x* and *a* being the last and first characters of the prefix and suffix
- *u* and *v* being the rest of the prefix and suffix
- *q* being our old state, *r* being our new one

Summary Definition: Turing Machine

A **Turing machine** is a 7-tuple
$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ with:

- Q as a finite set of states,
- Σ as the input alphabet not containing the blank symbol $\square$
- Γ as the tape alphabet, which is a superset of Σ and *does* contain $\square$
- δ: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ as the transition function,
- q_0 as the start state,
- q_{accept} as the accept state, and
- q_{reject} as the reject state, with $q_{accept} \neq q_{reject}$.

The heart of the machine is the transition function

- Say that $\delta(q, a) = (r, b, L)$
- Then on a state q and tape square containing symbol a, M will replace the a with b, transition to r, and move the tape head left

To represent the configuration with tape contents uv, in state q, with the head over the first symbol of v, we write: ***u q v***

ux q av yields:

- *u r xbv* if $\delta(q, a) = (r, b, L)$
- *uxb r v* if $\delta(q, a) = (r, b, R)$

Turing Acceptance

A Turing machine *M* **accepts** input *w* exactly as you'd expect: if there is a configuration sequence C_1, C_2, …, C_k so that:

1. C_1 is the start configuration of *M* on *w*,
2. Each C_i yields C_{i+1}
3. C_k is an accepting configuration

Now, the obvious definition

- The **language *L*(*M*) of a Turing machine** *M* is the set of strings that it accepts

…but now, some less obvious ones

- A Turing machine **recognizes** a language if it accepts the strings in it and no others
- A Turing machine **decides** a language if it accepts the strings in it ***and rejects all others***
- A language is **Turing-recognizable** if some Turing machine recognizes it
- A language is **Turing-decidable** if some Turing machine decides it

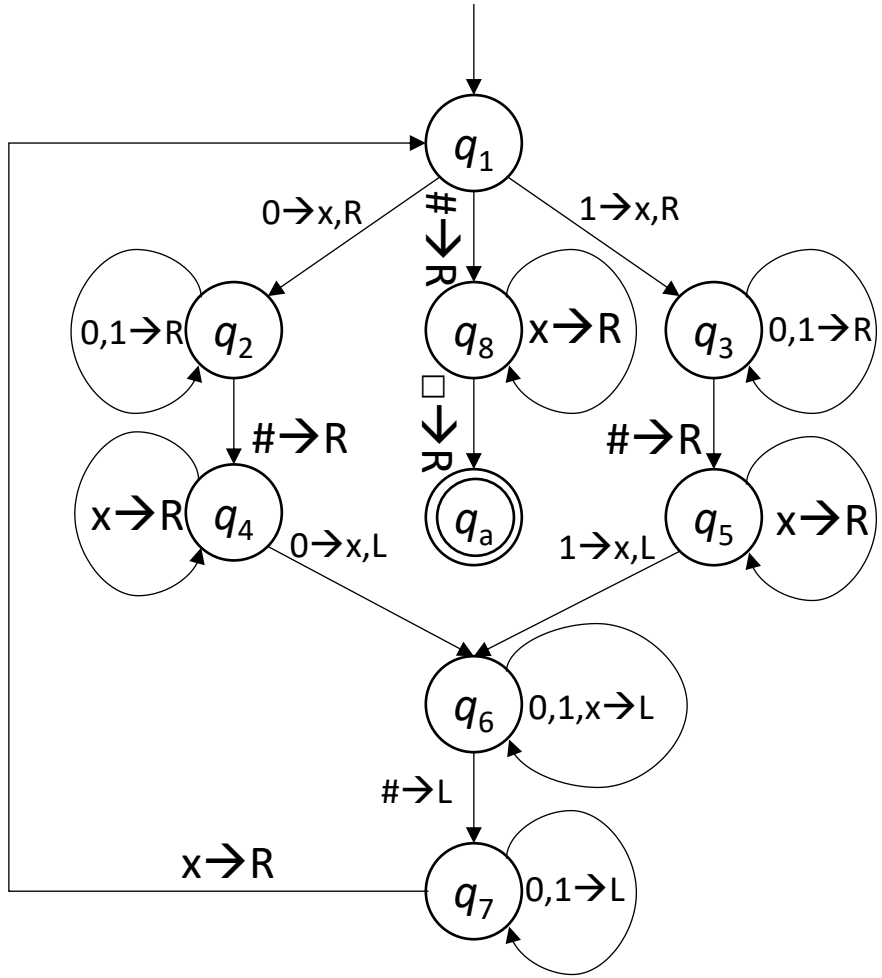
NOT ALL TURING-RECOGNIZABLE LANGUAGES ARE TURING-DECIDABLE!

Talking About Turing Machines

A Fully Formalized Turing Machine

Here's the formal definition and state machine for our { *w*#*w* | *w* in { **0**, **1** }* } decider.

- $Q = \{q_1, \ldots, q_8, q_{accept}, q_{reject}\}$
- $\Sigma = \{0, 1, \#\}$; $\Gamma = \{0, 1, \#, x, \square\}$
- $q_0 = q_1$
- q_{accept} shown as q_a
- q_{reject} as implicit – if we hit a symbol for which we do not have a transition, we move to q_{reject}
- δ as shown here to the right



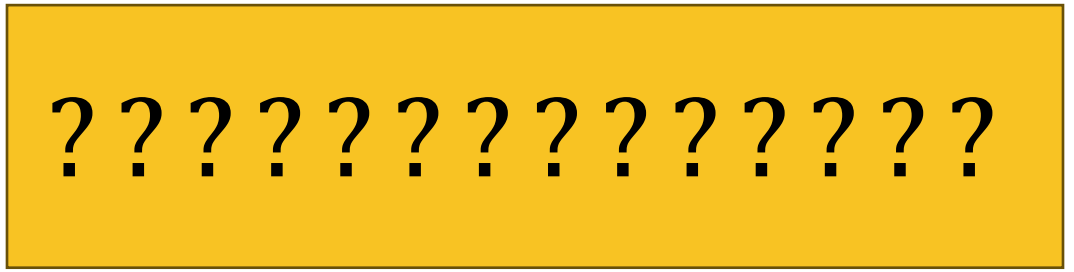
Remember: This is an *unusually simple* Turing machine.

Another Less Formal Turing Machine

On input string *w*:

1. Sweep left to right across the tape, crossing off every other 0.
   a. Keep track of whether there is more than one 0, and whether the number of 0's is even or odd.
2. Switch on what we kept track of about the number of 0s.
   a. If there is only one, **accept immediately and halt.**
   b. If there are an odd number other than one, **reject immediately and halt.**
3. Return the head to the leftmost square.
4. Return to step 1.

Can you tell what language this is accepting?



Another Less Formal Turing Machine

On input string *w*:

1. Sweep left to right across the tape, crossing off every other 0.
   a. Keep track of whether there is more than one 0, and whether the number of 0's is even or odd.
2. Switch on what we kept track of about the number of 0s.
   a. If there is only one, **accept immediately and halt.**
   b. If there are an odd number other than one, **reject immediately and halt.**
3. Return the head to the leftmost square.
4. Return to step 1.

Can you tell what language this is accepting?

$$L = \{0^{2^n} \mid n \geq 0\}$$

Levels of Formality in Discussing Turing Machines

- We've come to an interesting point in terms of formality.
- Turing machines are sufficiently complicated that quasi-formal descriptions are typically easier to understand *in every way* than full formalizations.
- Because of that, we're basically done with full formalizations. We're not going to do them any more, and unless we require it of you specifically, you don't have to either.
- These quasi-formal descriptions of Turing machines, or **implementation descriptions**, describe **what the machine does in response to the inputs,** with respect to how it moves its head and how it stores data on its tape.
- They're just shorthand for the full formal descriptions – we could turn any of them *into* those full formal descriptions, in fact, as long as we don't describe anything a Turing machine can't do.
- That's less of a problem than it sounds like. If you're worried about whether these levels of formality are really equivalent, don't be. We'll explain just how equivalent they are shortly. In fact, we'll go a good bit farther than that. But first, let’s talk about some variant Turing machines.

Acknowledgement

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3rd ed., 2013